Pack 06 · Model internals

The transformer details interviewers probe: attention math, masking, the KV cache, sampling and normalization. Lessons 05, 07, 10–14, 17. Answer each question out loud in about a minute, then check the card.

Q1 · Why ÷ √d, and why subtract the max in softmax? PRIORITY

Lessons: 05, 11

> **INTERVIEWER**
>
> Attention computes q · k ÷ √d before softmax. What goes wrong without ÷ √d (d = 128)? Softmax kernels subtract the row's maximum before e ^ x: why, and does it change the result?

60-second answer

- **Why scores grow with d:** a dot product of two d-dimensional vectors with unit-variance entries is a sum of d terms, so its variance is d and its spread is √d. For d = 128 that's √128 ≈ **11**.
- **Without ÷ √d:** scores that spread by about 11 make softmax nearly **one-hot**, which means attention locks onto a single token and the gradients vanish.
- **What ÷ √d does:** dividing by √d brings the spread back to about 1. It only rescales the spread; it doesn't center anything.
- **Why subtract the max:** e ^ x **overflows** quickly in FP16, because the largest FP16 value is 65,504, so even e ^ 12 is already too big. Subtracting the row's max makes every exponent ≤ 0, so every e ^ x is at most 1 and can't overflow.
- **It doesn't change the result:** softmax(x − c) = softmax(x), because subtracting c multiplies every term, top and bottom, by e ^ (−c), and that factor cancels.

Key numbers: $\text{softmax}(x)_i = e^{(x_i - \max)} \div \Sigma_j\, e^{(x_j - \max)}$.

Likely follow-up: "FlashAttention sees scores in blocks. How does it get an exact softmax?" It uses online softmax: it keeps a running max and a running sum as each block arrives. When a new block raises the max, it rescales the old sum and the output accumulator by e ^ (m_old − m_new), so the result is the same as if it had used the final max from the start.

> **TRAP**
>
> I said √d "centers the mean". In the online-softmax code I forgot the running max must never decrease, and forgot to rescale the accumulator.

Q2 · The causal mask, and why the KV cache is valid SOLID

Lessons: 11, 14, 17

> **INTERVIEWER**
>
> Future positions are masked by adding $-\infty$ before softmax: why $-\infty$ and not 0? Why do earlier rows of the score grid never need recomputing in decode, and how much work does the naive "re-feed everything" loop waste for a 512-token prompt and 256-token reply?

60-second answer

- **Why $-\infty$ and not 0:** the mask is added before softmax, and e^(−∞) = 0, so a masked token gets exactly zero weight. Adding 0 instead would give $e^0 = 1$, which is a full weight on the future token.
- **Why earlier rows never change:** a new token sits in the **masked future** of every earlier row, so those rows never see it. That means the earlier rows, and the K and V computed for earlier tokens, **never change**, which is why caching K and V is exact.
- **Cost of the naive loop:** each of the $N = 256$ steps re-feeds the $P = 512$ prompt tokens plus every token generated so far, so the work is $N \times P + N(N-1) \div 2 = 256 \times 512 + 32{,}640 = 131{,}072 + 32{,}640 \approx$ **164k token passes**.
- **What's actually needed:** each token only has to be processed once, so the work is $P + N = 512 + 256 =$ **768** passes. Since $164k \div 768 \approx 213$, the naive loop wastes about **213×**.
- **What the cache buys:** the KV cache turns each decode step into one new token attending to the cached past, instead of recomputing the whole sequence.

Key numbers: naive work $= N \times P + N(N-1) \div 2$; needed $= P + N$.

Likely follow-up: "What does the cache cost?" It costs memory: 128 KiB per token for Llama 3 8B. That memory is what limits how many users can run at once (pack 02).

> **TRAP**
>
> Give the reason, not just the fact: $e^0 = 1$ for why 0 fails, and "the new token is in their masked future" for why old rows stay valid.

Q3 · Three KV cache bugs, and a broken decode loop PRIORITY

Lessons: 10, 13, 17

> **INTERVIEWER**
>
> A preallocated KV cache shows three bugs: (1) the first token is fine, the rest gibberish; (2) outputs are subtly worse, with the zero-filled buffer read in full; (3) an answer mentions the previous user's conversation. And a decode loop uses `out.logits[:, 0, :].argmax(-1)` and ignores the prompt. What's wrong in each?

60-second answer

- **Bug 1, wrong position:** the decode tokens are encoded at the wrong position, because the RoPE position isn't advanced for each new token. Prefill used the right positions, so the first token comes out fine and everything after it is gibberish.

- **Bug 2, unfilled slots:** attention reads the empty, zero-filled slots of the buffer, and a zero key still gets some weight after softmax, so the output is subtly worse. The fix is to attend only to the filled length.
- **Bug 3, no reset:** the cache isn't cleared between requests, so a new request attends to the previous user's tokens. That makes it a correctness bug and also a **privacy** bug.
- **Decode loop:** the logits have shape **(batch, T, vocab)**, and position 0 predicts what follows the first token, so the loop ignores the rest of the prompt. The fix is `logits[:, -1, :]`, which samples from the last position.
- **Why engines compute only the last row:** full FP32 logits for 8,192 tokens are 8,192 × 128,256 × 4 B ≈ **4.2 GB**, while one row is 128,256 × 4 B ≈ **0.5 MB**. Only the last row is needed to pick the next token, so computing the rest is pure waste.

Key numbers: logits shape (batch, T, vocab); sample from the last position.

Likely follow-up: "How would you test a KV cache?" Run the cached loop and the naive loop on the same prompt and compare their outputs token by token. They must match exactly, because caching K and V is exact.

> **TRAP**
> I guessed "wrong K/V stored" for bug 1 (that would break the first decoded token too), and said logits are hidden × vocab (that's the LM head's weight matrix).

Q4 · Repeating stories vs bizarre words PRIORITY

Lessons: 13

> **INTERVIEWER**
> A story assistant at temperature 0 repeats the same phrase in long stories. Switching to temperature 1.2 with no other settings gives occasional bizarre words. Cause and fix for each?

60-second answer

- **Repetition at temperature 0:** temperature 0 is **greedy** decoding, which always takes the top token. So once a phrase becomes the most likely continuation, the model keeps choosing it and loops.
- **Fix for repetition:** sample instead, with a temperature around 0.7–1.0, and optionally add a repetition penalty.
- **Bizarre words at 1.2:** a **temperature above 1 flattens** the distribution, which lifts the long tail of junk tokens. With no other settings, nothing removes that tail, so junk tokens get drawn now and then.
- **Fix for junk:** cut the tail with **top-p**, which keeps the smallest set of tokens reaching probability p, or with **min-p**, which drops tokens below a fraction of the top token's probability.
- **Order matters:** apply **temperature first, then softmax, then cut the tail, then renormalize, then draw**.

Key numbers: p ∝ e^(logit ÷ T); T → 0 is greedy.

Likely follow-up: "Why is min-p often better than top-p at high temperature?" Because min-p's cutoff scales with the top token's confidence. So it cuts hard when the model is sure, and stays open when it isn't.

> **TRAP**
>
> I said "cut the tail, then apply temperature" (it's the other way round) and that temperature equalizes outcomes (it flattens).

Q5 · The residual stream and what RMSNorm normalizes PRIORITY

Lessons: 12

> **INTERVIEWER**
>
> Why does each layer do $x \leftarrow x + f(x)$ instead of $x \leftarrow f(x)$? Does RMSNorm normalize the residual stream itself, and why does that matter?

60-second answer

- **Why x + f(x):** the residual connection gives an **identity path**, so gradients flow straight through deep stacks. That means each layer only has to add a refinement on top of x instead of rebuilding it.
- **What gets normalized:** modern models are **pre-norm**, so each layer computes $x \leftarrow x + f(\text{RMSNorm}(x))$. RMSNorm normalizes only the **copy fed into each sublayer**, which means the stream itself stays raw and keeps accumulating.
- **What RMSNorm does:** it divides by the root mean square and applies a learned gain, with **no mean subtraction**. Skipping the mean is what makes it cheaper than LayerNorm.

Key numbers: $\text{RMSNorm}(x) = x \div \sqrt{(\text{mean}(x^2) + \varepsilon)} \times g$.

Likely follow-up: "Where does the final norm go?" There's one last RMSNorm right before the LM head, because the raw stream has grown through all the layers and has never been normalized itself.

> **TRAP**
>
> I didn't recall that only the sublayer input is normalized, not the stream.

Build it yourself

- `exercises/05_07_online_softmax.py`: exact softmax computed block by block, then the attention output in one pass, as FlashAttention does. Run `uv run exercises/05_07_online_softmax.py`.
- `exercises/10_13_sampler.py`: a sampler from raw logits (greedy, temperature, top-p, min-p, renormalize, draw). Run `uv run exercises/10_13_sampler.py`.
- `exercises/14_17_kv_cache.py`: a cached attention loop with preallocated buffers, proven to match the naive loop exactly, plus a demo of the unfilled-slot bug. Run `uv run exercises/14_17_kv_cache.py`.